

Don't Block Your Own Exit: Engineering a Reliable Anytime Solver for the OGC 2026 Grand Shipyard Challenge

Kevin Yin

August 15, 2026

ABSTRACT

The OGC 2026 Grand Shipyard challenge coupled irregular multilayer packing, temporal residence, bay assignment, weighted tardiness, workload balance, bay preferences, and directional crane precedence under a four-core hard deadline. The engineering problem was therefore larger than either scheduling or packing: a temporally attractive calendar could be geometrically unrealizable, a collision-free layout could violate crane access, and a strong search trajectory was worthless if it failed to return a checked solution in time. This report reconstructs a four-core anytime solver that selects between two fixed search topologies, accepts only complete current-run candidates, and preserves an independently checkable incumbent before the deadline. Across ten chronological organizer evaluations, all 60 hidden outputs were feasible. The report emphasizes the evidence that changed the architecture, the negative results that prevented wasted implementation, and the reliability controls required to make heuristic improvement usable. Cross-instance raw sums are reported later only as descriptive analysis, not as competition score.

Contents

Project at a glance

1. One realization problem, not two independent subproblems

- 1.1 Decision surface
- 1.2 Why schedule-first decomposition repeatedly broke
- 1.3 Reliability consumed optimization budget

2. Final solver architecture

- 2.1 Structural regime gate
- 2.2 Protected feasible path
- 2.3 Scored constructor
- 2.4 Calendar solver
- 2.5 Joint space-time repair
- 2.6 Atomic incumbent protocol and checker boundary
- 2.7 Native geometry kernel

3. Evidence that changed the architecture

- 3.1 Feasibility before optimization
- 3.2 Geometry became the hot path
- 3.3 Timing-only optimization did not ensure realization
- 3.4 In-memory success was not durable evidence
- 3.5 Supplied-instance improvement did not guarantee hidden transfer
- 3.6 More workers could reduce value
- 3.7 Opportunity preceded efficacy

4. Experimental method

- 4.1 Identity before comparison
- 4.2 Current-run versus historical evidence
- 4.3 Paired and order-balanced controls
- 4.4 Opportunity-first gates and independent review

5. Official evaluation history

- 5.1 Evidence correction
- 5.2 Ten-evaluation record
- 5.3 What the corrected trajectory shows

6. Contributions and AI-assisted development

- 6.1 Kevin Yin's role
- 6.2 AI assistance
- 6.3 Organizer-provided authority

7. Public source and validation boundary

8. Generalizable lessons

Conclusion

Provenance appendix

References

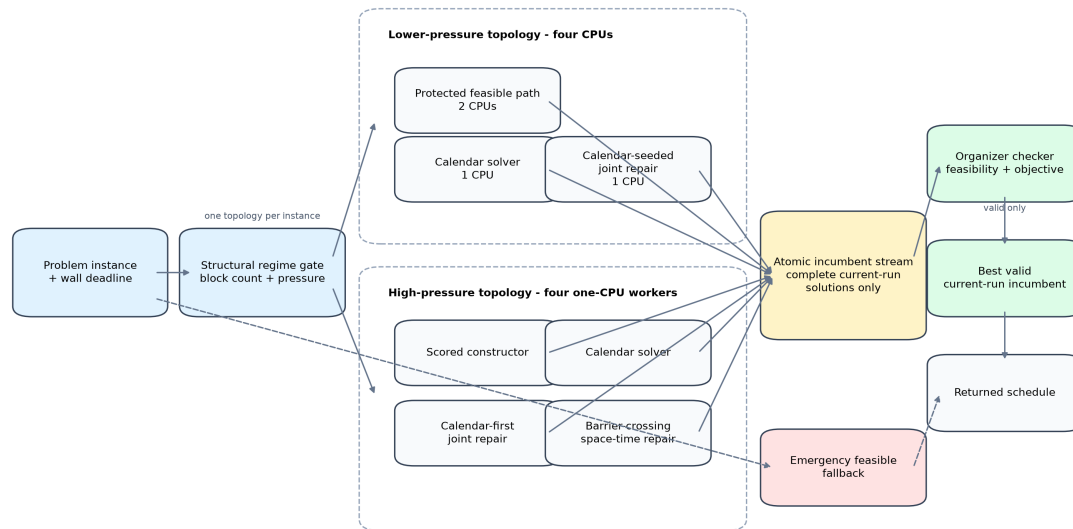
Project at a glance

Item	Record
Competition	Optimization Grand Challenge 2026, Grand Shipyard
Participant	Kevin Yin, Team Smoop
Problem class	Coupled irregular packing, temporal scheduling, and crane-access realization
Compute envelope	CPU-only, four cores, 16 GB memory, approximately 60 seconds per instance
Final architecture	Regime-gated current-run portfolio with a protected feasible path and checker-gated selection
Official campaign record	Ten chronological evaluations; 60/60 hidden outputs feasible
Final official vector	11,280; 31,368; 103,065; 6,464,170; 16,443,998; 42,333,422
Public release	Final Python source, native-kernel source, report, corrected result history, and reproducibility tests

The competition imposed a first-order feasibility requirement: an infeasible output, crash, process failure, or timeout received a score of -1 (OGC 2026 Organizing Committee 2026). That penalty changed what counted as an optimization algorithm. The useful product was not merely a search procedure that could occasionally find a superior schedule. It was an **anytime feasible system** that could search aggressively while preserving a validated return path.

The public artifact is deliberately narrower than the private campaign archive. It contains the final technical account, corrected official result history, selected methodology, frozen source, and reproducibility boundaries. It excludes organizer data and checker code, correspondence, raw experiment corpora, the certificate, the exact submitted ZIP, and the compiled submitted extension.

Final solver architecture: two mutually exclusive CPU topologies



Final solver architecture.

1. One realization problem, not two independent subproblems

1.1 Decision surface

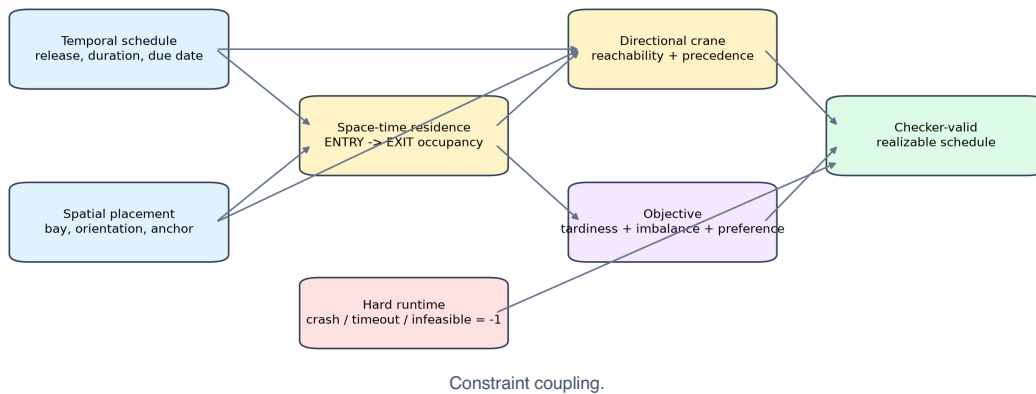
Each block required a bay, orientation, integer anchor, entry day, exit day, and operation order. The selected interval determined when the block occupied shipyard space. Same-day operations also had to respect directional crane access and precedence; when EXIT and ENTRY operations shared a day, EXIT operations were ordered first. The objective combined:

1. weighted tardiness;
2. workload imbalance across bays;
3. loss relative to bay preferences.

The challenge semantics therefore coupled time, placement, and operation order in one feasibility test. A low-tardiness calendar could become impossible when several blocks competed for the same physical cells. A collision-free static arrangement could become invalid when residence intervals overlapped. A schedule and layout that looked acceptable under separate partial checks could still violate crane reachability or same-day operation order.

Irregular-packing work commonly relies on no-fit representations, rasterized occupancy, discrete anchor models, coordinate descent, and exact-heuristic hybrids (Toledo et al. 2013; Mundim et al. 2017, 2018; Umetani and Murakami 2022; Gomes and Oliveira 2006). The Grand Shipyard challenge added a temporal and operational layer. Work on spatial shipbuilding schedules and block relocation reinforces why placement cannot be optimized independently of future access and operational sequence (Ge and Wang 2021; Kim and Hong 2006).

Why a good calendar can still fail in the yard



1.2 Why schedule-first decomposition repeatedly broke

A tempting pipeline is:

optimize calendar -> place blocks -> repair collisions -> return

That pipeline was useful as a constructor, but it was insufficient as a complete search model in this implementation. Timing-only experiments produced lower-cost calendar states that could not survive the canonical realization/checker path; the failure was therefore a design observation from this project, not a proof that all decomposed methods are inferior. The decoder still had to answer a joint question:

Can this bay, date, orientation, anchor, residence interval, and operation order coexist with the frozen outside state and remain crane-realizable?

When the answer was no, changing only one coordinate often preserved the actual cause of failure. Delaying a block could create new tardiness without opening a valid spatial route. Moving a block could improve placement but create a later crane obstruction. Reassigning a bay could improve preference while worsening the load-range term. The later solver therefore treated a placement as a **space-time column**, not as an isolated spatial coordinate.

1.3 Reliability consumed optimization budget

The hard deadline made process behavior part of the algorithm. CPU allocation, serialization, validation, signal handling, cleanup, and fallback reserves all consumed time that could otherwise have gone to search. But eliminating those reserves would have made the objective comparison meaningless: an unreturned or invalid improvement had zero competitive value.

The final design therefore separated two states:

- a mutable working state that could traverse neutral or temporarily worse configurations; and
- an immutable best-valid incumbent that remained independently returnable.

The practical consequence was to preserve enough time and state for the solver to finish its own work: validate the best available candidate, terminate descendants, clean up its process tree, and return before the deadline.

2. Final solver architecture

The final solver was deterministic at the orchestration level given a completed structural-feature estimate, and current-run at the evidence level. It did not retrieve historical hidden solutions. Every worker solved the current input, published only complete candidates, and competed through a small durable incumbent protocol. A parent process returned only a candidate that passed the organizer-provided checker.

2.1 Structural regime gate

The parent derived two cheap structural features: block count and an offered-load pressure ratio

```
pressure = sum_i(min_orientation_area_i * processing_time_i)
           / (total_bay_area * (max_due_date - min_release_time))
```

The high-pressure topology was eligible only when `pressure >= 0.70` and the instance contained at least 250 blocks; otherwise the lower-pressure topology was used. A memory estimate could also force the lower-pressure path. These were frozen deployment thresholds derived from the project's supplied-instance measurements. They are heuristic deployment rules rather than a learned selector; the hidden evaluation does not isolate the gate's causal contribution. The gate chooses between two fixed CPU configurations whose contention behavior had been measured locally.

Lower-pressure topology

- protected feasible path using two CPUs;
- calendar solver using one CPU;
- calendar-seeded joint-repair worker using one CPU.

High-pressure topology

- scored constructor using one CPU;
- calendar solver using one CPU;
- calendar-first joint-repair worker using one CPU;
- calendar-and-geometry causal-repair worker using one CPU.

This correction matters conceptually: the final solver did **not** run all five roles simultaneously. It selected one four-core topology per instance.

2.2 Protected feasible path

The lower-pressure topology retained a two-core path descended from the strongest previously organizer-tested solver. Its primary role was to preserve a validated solution trajectory while other workers attempted more structural changes. Constructive placement, cached forbidden maps, bounded local search, and defensive deadline handling kept an incumbent accessible even after exceptions or late alarms.

The path's internal validation was not accepted on trust. Its output still passed through the same parent-level checker boundary as every other worker proposal.

2.3 Scored constructor

In the high-pressure topology, a one-core constructor emphasized objective-bearing feasibility rather than simply placing the easiest next block. Its ordering and candidate scoring combined release dates, durations, due dates, bay preferences, workload effects, and spatial availability. It streamed improving complete incumbents during execution instead of publishing only its terminal state.

Streaming mattered because process termination near the deadline was expected. A worker could be stopped after publishing a useful current-run solution without losing all of its work.

2.4 Calendar solver

The calendar lane produced a temporal assignment before full geometric realization. It could expose lower-tardiness temporal structure that a placement-first policy never visited. The calendar itself was never treated as a solution. It published a complete realized candidate when possible and exposed a bounded calendar channel for a repair consumer.

That channel was separate from the global incumbent. A calendar candidate could be worse than the current best complete schedule and still contain a useful temporal seed for joint repair.

2.5 Joint space-time repair

The repair lane replayed a current complete solution into a mutable model, removed a coherent set of blocks, and enumerated concrete space-time alternatives against the frozen outside background. A single-threaded CP-SAT subproblem then chose exactly one alternative column per removed block, prohibited pairwise-conflicting columns and prior no-good combinations, and minimized the weighted tardiness, preference, and bay-load imbalance term over that bounded neighborhood. The materialized result was retained only after a strict checker-valid improvement.

The neighborhood was causal rather than purely geometric. Blocks could be selected because they contributed to tardiness, preference loss, spatial obstruction, or a calendar-realization failure. The exact subproblem coordinated a bounded set of alternatives that sequential regret repair could miss.

The high-pressure calendar-and-geometry repair worker began from a coupled calendar/placement seed and then applied causal joint repair. Its working state could cross neutral or temporarily worse arrangements while the returned best remained protected. This allowed search across disconnected basins without exposing an inferior return.

2.6 Atomic incumbent protocol and checker boundary

Workers published records containing:

```
schema version
source role
generation
objective
complete solution
metadata
```

Writes used same-directory temporary files, `fsync`, and atomic replacement. A stable companion lock serialized publishers. A record replaced the incumbent only when its objective was strictly lower, except for a narrow equal-objective case that replaced a partial record with a final validated record.

The protocol rejected nonfinite objectives, corrupt JSON, and incomplete solution records. It supported recovery after a partial file and retained a generation history during private testing.

The parent did not infer feasibility from worker identity. It loaded each candidate and called the organizer checker. A proposal without a valid objective or with any feasibility failure was discarded. The parent selected the lowest objective among valid current-run candidates.

```
worker proposal -> durable record -> organizer checker -> parent selection
```

An emergency path generated or recovered a feasible fallback if ordinary workers failed to publish in time. Deadline reserves protected final parsing, checking, selection, and process cleanup.

2.7 Native geometry kernel

The hottest primitive constructed a feasible anchor map from multilayer block masks and forbidden occupancy. The submitted solver used a CPython 3.12 Linux extension. The public source release includes a C++17/pybind11 implementation that packs each column into a 32-bit bitset, shifts masks over candidate anchors, and returns a Boolean feasible map.

The native path accelerated an existing representation; it did not change challenge semantics. A Python fallback remained available when the extension could not load. The public tests build the source and compare it against a direct NumPy reference on randomized multilayer cases.

3. Evidence that changed the architecture

The development history is more useful as a sequence of causal corrections than as a list of internal candidate numbers.

3.1 Feasibility before optimization

The first objective was to produce complete schedules consistently. Early work established:

- deterministic constructive ordering;
- exact checker integration;
- module-level incumbent protection;
- wall-clock alarms and deadline reserves;
- clean fallback behavior;
- reproducible release packaging.

The first two official evaluations established a stable operational baseline. The second improved one hidden instance while leaving the other five unchanged; the third reproduced the second vector exactly. That repeat mattered because later changes could be compared against a controlled deployment path rather than against an unstable packaging process.

3.2 Geometry became the hot path

Repeated containment and collision work limited how many constructive and repair decisions could be completed before the deadline. The system moved toward cached forbidden maps, compact occupancy operations, and a compiled anchor-feasibility primitive.

The important measurement was not primitive speed in isolation. A faster predicate had value only if it produced more completed search work or a better valid incumbent before the same wall deadline. Several speedups were rejected because they did not change the useful unit of work or changed it in a way that degraded objective quality.

The first compiled-geometry official evaluation materially improved P3-P6. A later versioned forbidden-map cache was followed by another broad P3-P6 improvement in the official evaluation. Direct organizer-email reconciliation shows that this cache-era jump was much larger than an earlier screenshot transcription had suggested.

3.3 Timing-only optimization did not ensure realization

A low-tardiness calendar was not equivalent to a realizable schedule. The decoder could lose its temporal gain when it encountered occupancy or crane conflicts. That failure shifted development away from deeper optimization of an unrealizable calendar alone and toward joint space-time columns, calendar-seeded repair, and bounded exact coordination.

3.4 In-memory success was not durable evidence

One earliest-entry investigation produced promising in-process behavior but could not reproduce the required state from persisted artifacts. The result was invalidated rather than promoted. The evidence rule changed: a mechanism had to survive exact serialization, reload, replay, and checker validation.

This was not paperwork. The final solver itself depended on multiple processes communicating through files near a hard deadline. The persistence layer was part of the algorithmic surface.

3.5 Supplied-instance improvement did not guarantee hidden transfer

Here, **supplied-instance evidence** means organizer-provided development instances available before submission; it is distinct from the six hidden evaluation cells. A topology-based repair version produced strong supplied-instance evidence and was reasonable to submit on the information available at the time. Its official hidden evaluation remained feasible but regressed on P3, P4, and P6 relative to the preceding protected-floor version.

One counterexample was enough to reject the stronger operational assumption that broad supplied-instance wins guaranteed hidden dominance. It did **not** identify a statistical “transfer ceiling,” prove distribution shift, or distinguish overfitting from runtime variance. The later report and release therefore distinguish:

- public-instance evidence;
- official hidden evidence;
- source identity;
- runtime outcome;
- historical stored output;
- current-run result.

This episode prevented an attractive public-success narrative from overriding the external evaluation.

3.6 More workers could reduce value

A fourth lane sometimes consumed CPU that the protected feasible path used more effectively. Post-hoc improvement of a frozen floor did not imply a win against the still-running floor under equal wall time. Unrestricted concurrent racing produced deep-instance upside but shallow-instance contention.

The consequence was the structural regime gate: the final architecture used a four-way one-core portfolio only where pressure features indicated complementary value. Elsewhere, the protected path retained two CPUs.

3.7 Opportunity preceded efficacy

Several plausible mechanisms failed before a meaningful efficacy test because the required event did not occur often enough under their frozen opportunity gates. Examples included 0 specialist-eligible requests against a minimum of 3, 0 exact cycle conflicts against a minimum of 2, and a transactional replay gate whose 24/24 qualification executions were checker-feasible but retained 0/128 eligible states. Other gates found no cross-worker import opportunity, no remote-specific redirect, or insufficient complete-counterfactual action coverage.

These were negative results about **engagement**, not broad refutations of the underlying research ideas. The thresholds were declared before the measured gate; the accepted conclusion was only that the current solver did not expose enough of the required event for those specific mechanisms to matter.

4. Experimental method

4.1 Identity before comparison

Every release candidate was bound to source hashes, package members, and a release commit. Deterministic ZIP construction fixed member order, timestamps, modes, and compression. A build refused missing members, extra files, or source-hash drift.

This prevented a common heuristic-search failure: comparing a result to a candidate name when the actual bytes had changed.

4.2 Current-run versus historical evidence

A stored output could prove that a mechanism had once produced a result. It could not prove current runtime reliability, current package identity, or current process interaction. Promotion gates therefore distinguished:

- historical best output;
- in-process current output;
- persisted current output;
- exact packaged output.

The final parent selected only current-run candidates.

4.3 Paired and order-balanced controls

Fixed-wall stochastic search varied even with stable source. One controlled same-binary, back-to-back run of the same solver differed by 1.446% in objective; that is one observed instability case, not a global noise-rate estimate. Where attribution mattered, experiments therefore used paired controls, reversed order, repeated same-binary runs, or explicit noise envelopes. The purpose was not to eliminate all runtime variance. It was to avoid attributing an ordinary trajectory fluctuation to a code change.

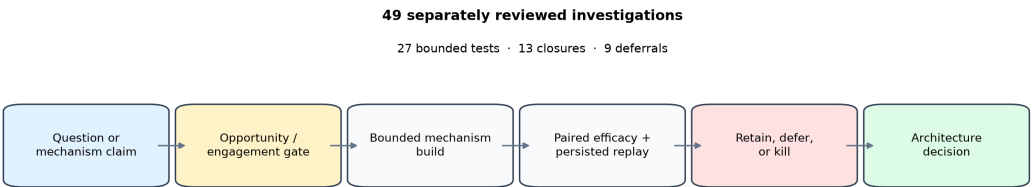
Random restarts and portfolio allocation are well established responses to heavy-tailed or erratic search behavior (Luby et al. 1993; Gomes et al. 2000; Fischetti and Monaci 2014; Weise et al. 2019). In this project, however, a portfolio was justified only when each lane produced complementary current-run value after contention.

4.4 Opportunity-first gates and independent review

Before building an expensive treatment, the campaign often tested whether its required event existed. Examples included specialist eligibility, exact reusable cycle conflicts, publish/read/import events between workers, remote-specific redirects, complete action coverage, and second transactional states.

The formal research program recorded 49 separately reviewed investigations. In that workflow, one investigation bound a research question to one specialist response, a provenance record, and a separate review record; the recommendations were 27 bounded tests, 13 closures, and 9 deferrals. Six selected opportunity-first mechanisms reached terminal kills under their frozen gates. The counts describe the research-control process, not independent experimental replicates or evidence of solver quality.

Research gate: prove opportunity before efficacy



Six selected opportunity-first gates reached valid terminal kills

The terminal sprint used a lower-latency workflow when formal coordination cost became dominant.

Research and validation flow.

The formal workflow was strong at pruning and provenance but became too coordination-heavy for the last competition window. The terminal sprint moved to a lower-latency direct-research workflow. That is itself a process result: review rigor has value, but its latency must remain below the decision horizon.

5. Official evaluation history

5.1 Evidence correction

This v1.1 report corrects a load-bearing data defect in public v1.0. The earlier release inherited an unsupported historical matrix, and a screenshot-based private transcription for the fifth evaluation conflicted with the complete organizer email. The ten rows below were rebuilt from the completed-evaluation emails retained in the registered Team Smoop mailbox.

The final official vector, the 60/60 feasibility record, and the 28.316% first-to-final descriptive reduction were already correct. The correction changes the intermediate trajectory and the attribution of several improvements.

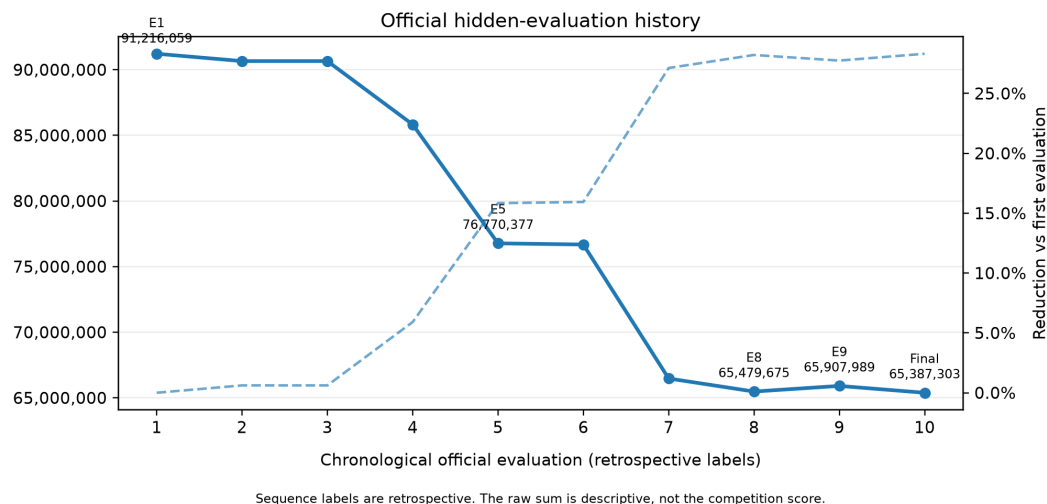
Ten successive solver versions received organizer evaluation on the same six hidden evaluation cells. The sequence numbers below are **chronological labels used by this retrospective**, not **organizer-assigned submission identifiers**. The organizer-issued result emails are retained privately; the public release transcribes those official results and binds the transcription by hash.

5.2 Ten-evaluation record

Eval.	P1	P2	P3	P4	P5	P6	Descriptive raw sum
1	11,280	31,368	130,705	10,153,470	28,945,493	51,943,743	91,216,059
2	11,280	31,368	130,705	9,593,960	28,945,493	51,943,743	90,656,549
3	11,280	31,368	130,705	9,593,960	28,945,493	51,943,743	90,656,549
4	11,280	31,368	123,880	7,992,878	26,711,863	50,955,045	85,826,314
5	11,280	31,368	105,855	7,215,499	22,333,093	47,073,282	76,770,377
6	11,280	31,368	105,855	7,215,499	22,243,379	47,073,282	76,680,663
7	11,280	31,368	103,065	7,136,114	17,492,603	41,698,677	66,473,107
8	11,280	31,368	103,065	6,187,873	17,447,412	41,698,677	65,479,675
9	11,280	31,368	110,230	6,368,452	17,447,412	41,939,247	65,907,989
10	11,280	31,368	103,065	6,464,170	16,443,998	42,333,422	65,387,303

All ten result emails reported six feasible outputs. The campaign therefore ended with **60/60 feasible hidden-instance executions** and no observed crash, timeout, or infeasibility penalty.

The first-to-final descriptive raw sum decreased by **28.316%**. That aggregate is **not the competition score**. Official scoring ranked teams independently on each instance, so the competitive effect depended on field values and where gains or regressions occurred.



Official evaluation history.

5.3 What the corrected trajectory shows

The sequence below is an externally observed deployment history, **not an ablation study**. A change that precedes an official improvement is not, by that chronology alone, proven to have caused it. Mechanism-level causal claims in this report rely on separate local controls where available; the official rows establish only what each submitted package returned.

Four features are more informative than a monotone-success story.

First, the second evaluation improved only P4, and the third repeated it exactly. That establishes both a targeted hidden gain and deployment repeatability.

Second, the fourth and fifth evaluations followed releases dominated by geometry and cache work and each improved P3-P6; the fifth reduced the descriptive raw sum by 10.551% relative to the fourth. That chronology is consistent with the local controlled evidence for those mechanisms, but the hidden before/after rows do not isolate a single cause. The sixth then tied five instances and improved only P5. The prior transcription had incorrectly assigned much of the fifth evaluation's gain to the sixth.

Third, the seventh produced the largest later aggregate step, with broad P3-P6 improvements, and the eighth concentrated additional gain in P4 and P5. These rows are deployment outcomes; the stage names in the provenance appendix are retrospective lineage labels rather than causal assignments by the organizer.

Fourth, the ninth was the clearest supplied-to-hidden transfer counterexample: it regressed P3, P4, and P6 relative to the eighth. The final solver then improved P3 and P5 relative to the ninth while worsening P4 and P6. Its descriptive raw sum was the lowest of the ten, but it did not dominate the eighth instance by instance.

P1 and P2 were unchanged across all ten evaluations. The organizer result records do not diagnose why, so the report leaves that constancy uninterpreted.

6. Contributions and AI-assisted development

6.1 Kevin Yin's role

Kevin Yin retained responsibility for:

- decomposing the challenge into geometry, scheduling, crane, reliability, and search surfaces;
- selecting solver architectures and candidate lineages;
- defining research questions, experiment gates, promotion criteria, and rollback rules;
- interpreting local and official evidence;
- deciding which exact package to submit;
- deciding when to stop the campaign;
- approving the public claim boundary and final narrative.

6.2 AI assistance

AI systems were used throughout literature synthesis, source inspection, implementation, test construction, experiment-execution support, adversarial review, evidence normalization, and documentation. Responsibility is stated at the decision and process level rather than reduced to line-by-line authorship percentages.

Multiple agents and models were used to obtain independent perspectives, but model agreement was never treated as empirical validation. Accepted checker results, frozen artifacts, measured runs, and Kevin Yin's promotion/submission decisions remained the governing evidence.

The competition explicitly allowed AI-assisted development while assigning quality and correctness responsibility to participants. This report presents that assistance as part of the engineering process rather than hiding it or claiming autonomous authorship.

6.3 Organizer-provided authority

The challenge formulation, instances, official checker, evaluation environment, and hidden evaluations belonged to the organizer. The public repository does not redistribute those materials. Source files import the organizer's `utils` module when executed inside the challenge environment.

7. Public source and validation boundary

The public release provides **source traceability and component-level validation**. It contains the seven Python files included in the final submitted package, byte-identical to their frozen private versions, plus the C++/pybind11 geometry-kernel source and randomized equivalence tests.

The release is intentionally limited to participant-authored source and documentation. Organizer-owned challenge materials are not redistributed, and the private competition package, correspondence, credential, and raw experiment corpus remain in the private evidence archive. Exact private artifact identities are bound by SHA-256 in the source manifest. The native-kernel source can be rebuilt and behaviorally tested from the public repository.

The corrected result CSV is generated from an immutable in-repository matrix whose private authority is the direct organizer-email reconciliation. Consecutive asset and report builds are required to be byte-identical before release.

8. Generalizable lessons

1. **Feasibility is an architectural invariant.** It cannot be added as a final validation step to a search that routinely destroys its only return path.
2. **The useful unit is the coupled decision.** Optimizing schedule, geometry, or worker communication in isolation often changes no valid endpoint.
3. **Opportunity precedes efficacy.** Before building a sophisticated selector or communication channel,

prove that multiple meaningful actions occur often enough to justify it.

4. **Current-run evidence outranks stale excellence.** A historical output is not a current package, runtime, or cleanup guarantee.
5. **Negative experiments are capital.** A valid kill narrows the system and protects the terminal sprint from repeatedly attractive ideas.
6. **Reliability consumes budget.** Process startup, serialization, checking, cleanup, and fallback reserves are part of the optimization algorithm.
7. **Supplied-instance validation is evidence, not a guarantee.** Broad local wins can justify a submission, but they do not establish hidden superiority.
8. **Research governance has latency.** Heavy review can improve decisions while becoming unsuitable for a deadline-critical terminal phase.
9. **Evidence reconciliation must reach the direct source.** A polished report and passing test suite can still preserve a wrong historical matrix if they verify only internal consistency.

Conclusion

The Grand Shipyard campaign evolved from a feasible constructor into a checker-gated, current-run, multicore anytime system. The central technical shift was recognizing that scheduling, placement, residence, crane access, objective value, and runtime reliability formed one realization problem. The central engineering shift was protecting a returnable incumbent while allowing specialized workers to search more aggressively.

The final solver closed the campaign with six feasible official outputs, a 60/60 campaign feasibility record, and the lowest descriptive raw six-instance sum of the ten chronological evaluations. Its durable contribution is the architecture, the evidence discipline around it, and the negative results that made the final system smaller and more reliable.

The title is literal in more than one direction. A shipyard block can obstruct a future EXIT if placement ignores crane access. An optimization process can block its own exit too: aggressive search that destroys its only valid incumbent or consumes the time reserved for checking and cleanup has converted optional upside into operational failure. The final architecture was built to keep both exits clear—a realizable route for the blocks and an independently returnable route for the solver. Project closeout followed the same discipline: preserve the evidence, leave a clean handoff, and stop before continuation becomes its own obstruction.

Provenance appendix

Internal identifiers are retained here for exact traceability rather than used as the conceptual vocabulary of the main report.

Semantic stage	Internal development identity	Purpose
Joint space-time repair lineage	Candidate 11	Calendar-and-geometry repair and causal neighborhoods
Topology-based repair experiment	Candidate 14	Public-success / hidden-transfer counterexample
Current-run portfolio lineage	Candidate 19	Independent worker portfolio and incumbent protocol
Final solver	Candidate 23	Terminal frozen architecture; retrospective evaluation label 10

Artifact	Identity
Final private release commit	731b3ca9a59c898f4b097e95b75f43336ccb55ee
Final private release tag	candidate23-release-20260727
Private submission ZIP SHA-256	0b3442ebc2513cd0bedece780f33331f719293513667f075bb3cface08c4cf4e
Submitted native extension SHA-256	a1f0309bc7d30a6482528bf9a6b52107e02e622365a33c1517d65fbaa1636737
Private evidence-correction commit	7841de97e22ce753667abfa236f76836c57511d3
Private evidence-correction tag	ogc2026-evaluation-history-erratum-v1.0
Public release	v1.1.1

References

- Fischetti, Matteo, and Michele Monaci. 2014. "Exploiting Erraticism in Search." *Operations Research* 62 (1): 114–22. <https://doi.org/10.1287/opre.2013.1231>.
- Ge, Yan, and Aimin Wang. 2021. "Spatial Scheduling for Irregularly Shaped Blocks in Shipbuilding." *Computers & Industrial Engineering* 152: 106985. <https://doi.org/10.1016/j.cie.2020.106985>.
- Gomes, A. Miguel, and José F. Oliveira. 2006. "Solving Irregular Strip Packing Problems by Hybridising Simulated Annealing and Linear Programming." *European Journal of Operational Research* 171 (3): 811–29. <https://doi.org/10.1016/j.ejor.2004.09.008>.
- Gomes, Carla P., Bart Selman, Nuno Crato, and Henry Kautz. 2000. "Heavy-Tailed Phenomena in Satisfiability and Constraint Satisfaction Problems." *Journal of Automated Reasoning* 24 (1–2): 67–100. <https://doi.org/10.1023/A:1006314320276>.
- Kim, Kap Hwan, and Gyu-Pyo Hong. 2006. "A Heuristic Rule for Relocating Blocks." *Computers & Operations Research* 33 (4): 940–54. <https://doi.org/10.1016/j.cor.2004.08.005>.
- Luby, Michael, Alistair Sinclair, and David Zuckerman. 1993. "Optimal Speedup of Las Vegas Algorithms." *Proceedings of the 2nd Israel Symposium on Theory and Computing Systems (ISTCS)*, 128–33. <https://doi.org/10.1109/ISTCS.1993.253477>.
- Mundim, Leandro R., Marina Andretta, Maria Antónia Carravilla, and José F. Oliveira. 2018. "A General Heuristic for Two-Dimensional Nesting Problems with Limited-Size Containers." *International Journal of Production Research* 56 (1–2): 709–32. <https://doi.org/10.1080/00207543.2017.1394598>.
- Mundim, Leandro R., Marina Andretta, and Thiago Alves de Queiroz. 2017. "A Biased Random Key Genetic Algorithm for Open Dimension Nesting Problems Using No-Fit Raster." *Expert Systems with Applications* 81: 358–71. <https://doi.org/10.1016/j.eswa.2017.03.059>.
- OGC 2026 Organizing Committee. 2026. *OGC 2026: The Grand Shipyard Puzzle — Problem Statement*. Official participant document, version 1.2.
- Toledo, Franklina M. B., Maria Antónia Carravilla, Cristina Ribeiro, José F. Oliveira, and A. Miguel Gomes. 2013. "The Dotted-Board Model: A New MIP Model for Nesting Irregular Shapes." *International Journal of Production Economics* 145 (2): 478–87. <https://doi.org/10.1016/j.ijpe.2013.04.009>.
- Umetani, Shunji, and Shohei Murakami. 2022. "Coordinate Descent Heuristics for the Irregular Strip Packing Problem of Rasterized Shapes." *European Journal of Operational Research* 303 (3): 1009–26. <https://doi.org/10.1016/j.ejor.2022.03.034>.
- Weise, Thomas, Zijun Wu, and Markus Wagner. 2019. "An Improved Generic Bet-and-Run Strategy with Performance Prediction for Stochastic Local Search." *Proceedings of the AAAI Conference on Artificial Intelligence* 33: 2395–402. <https://doi.org/10.1609/aaai.v33i01.33012395>.